Module 03a — OOP Part 1: Classes and Encapsulation

Phase: Advanced Python
Previous: Modules and Packages
Next: OOP Part 2: Inheritance and Composition

Series: OOP overview · Part 1 (this file) · Part 2 · Part 3 · Part 4

Lab kit: 15_Lab_Assets/17_bank_account_oop/ — class warm-up Mini-Project below; full CLI is in Part 3.

Module Outcome

Turn scattered dicts and free functions into a class that owns its data, choose the right kind of variable and method, and stop outside code from breaking the rules.

By the end of this module you will be able to:

- Define a class, create objects, and use `self` and `__init__`
- Choose instance vs class vs local variables
- Write instance, class, and static methods
- Encapsulate state with `_`, `__`, and `@property` (lab kit may use `__balance` + `get_balance()` — both styles are acceptable)
- Build a `BankAccount` class (no menu yet)

1. Why This Concept Matters

A student tracker often starts as dicts plus free functions. Data lives in one place; behaviour lives somewhere else:

```
s1 = {"roll": 101, "name": "Anita", "marks": [72, 68, 81]}
s2 = {"roll": 102, "name": "Vikram", "marks": [40, 35, 48]}

def average(student):
    marks = student["marks"]
    return sum(marks) / len(marks)

def grade(student):
    avg = average(student)
    if avg >= 85:
        return "A"
    if avg >= 70:
        return "B"
    if avg >= 50:
        return "C"
    return "Fail"

def display(student):
    print(f"{student['roll']}: {student['name']} — {grade(student)}")

display(s1)
display(s2)
```

Expected Output:

```
101: Anita — B
102: Vikram — Fail
```

That style breaks as the program grows:

1. Anyone can corrupt the dict (`s1["marks"] = -10`, `del s2["name"]`).
2. Data and behaviour are **scattered** — easy to pass the wrong shape.
3. Adding `email` or `is_pass()` means hunting every function that touches a student.
4. The same mess repeats for Order, Account, Product.

Bundle **data + behaviour** into one unit — a **class**. Each filled form is an **object**:

```
class Student:
    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.marks = list(marks)   # own copy — don't share the caller's list

    def average(self):
        if not self.marks:
            return 0.0
        return sum(self.marks) / len(self.marks)

    def grade(self):
        avg = self.average()
        if avg >= 85:
            return "A"
        if avg >= 70:
            return "B"
        if avg >= 50:
            return "C"
        return "Fail"

    def display(self):
        print(f"{self.roll}: {self.name} — {self.grade()}")

s1 = Student(101, "Anita", [72, 68, 81])
s2 = Student(102, "Vikram", [40, 35, 48])
s1.display()
s2.display()
print("Anita average:", round(s1.average(), 1))
```

Expected Output:

```
101: Anita — B
102: Vikram — Fail
Anita average: 73.7
```

```
CLASS Student          OBJECT s1             OBJECT s2
roll, name, marks      Anita, 101            Vikram, 102
average / grade / display
```

That shift — from "bags of data + floating functions" to "objects that know how to behave" — is **Object-Oriented Programming (OOP)**.

This part focuses on **classes** and **encapsulation**. Inheritance is Part 2. Polymorphism is Part 3. Abstract classes are Part 4.

---

2. Class, object, `self`, `__init__`

Understand

A **class** is the blueprint (form template). An **object** is one filled form.

`__init__` runs when you create an object and sets starting data. `self` is "the object being built or used right now" (like `this` in Java — Python makes you write it).

On `s1.display()`, Python secretly passes `s1` as `self`.

How It Works

```
class Student:
    def __init__(self, roll, name):   # constructor
        self.roll = roll              # instance attribute
        self.name = name

    def display(self):                # instance method
        print(f"{self.roll}: {self.name}")

s1 = Student(101, "Anita")            # object creation
s2 = Student(102, "Vikram")
s1.display()
s2.display()
print("type(s1):", type(s1).__name__)
print("same class?", type(s1) is type(s2))
```

Expected Output:

```
101: Anita
102: Vikram
type(s1): Student
same class? True
```

What happens on `Student(101, "Anita")`:

1. Python creates a new empty object.
2. `__init__(self, 101, "Anita")` runs — `self` is that new object.
3. Attributes are attached to `self`.
4. The name `s1` now **refers** to that object.

	Method	Constructor (`__init__`)
Name	Any valid name	Always `__init__`
When it runs	When you call it	Automatically on `ClassName(...)`
How often per object	As many times as you call	Once (at creation)

	Method	Constructor (`__init__`)
Typical job	Business logic	Declare and initialize instance data
Return	Optional value	Should not return a useful value

`s1` is a **reference**. You rarely call `__init__` by name. If you omit `__init__`, Python still creates an empty object — usually a poor fit for real entities.

⚠️ WATCH OUT

```
# Wrong — forgot self in the method signature
class Student:
    def display():
        print("hi")

s1 = Student()
s1.display()    # TypeError: display() takes 0 positional arguments but 1 was given
```

Python always passes the object as the first argument of an instance method. Name it `self`.

Do not invent five classes for a ten-line script. Use functions until the data and rules belong together.

Try It

Create a `Product` class with `sku`, `name`, and `price`, plus a `label()` method that returns `sku: name @ ₹price`. Create two products and print both labels.

---

3. Instance, class, and local variables

Understand

Three lifetimes, three jobs:

Kind	Where	Who owns it
Instance	`self.emp_id`	One object
Class	`Employee.company`	Shared by all objects of the class
Local	`title = "Staff"` inside a method	Lives only while that call runs

Company name can be shared. Employee id must not. A temporary fee or title should not leak as an attribute.

How It Works

```
class Employee:
    company = "KSR Datavizon"   # class attribute — shared

    def __init__(self, emp_id, name):
        self.emp_id = emp_id    # instance attribute — unique per object
        self.name = name

    def label(self):
        title = "Staff"         # local — method only
        return f"{title}: {self.name} @ {Employee.company}"
```

```
e1 = Employee(1, "Ravi")
e2 = Employee(2, "Meera")
print(e1.label())
print(e2.label())
print("company via class:", Employee.company)

e1.company = "Acme"             # shadows on e1 only
print(e1.company)
print(e2.company)
print(Employee.company)
```

Expected Output:

```
Staff: Ravi @ KSR Datavizon
Staff: Meera @ KSR Datavizon
company via class: KSR Datavizon
Acme
KSR Datavizon
KSR Datavizon
```

To change the shared value for **everyone**, assign via the class: `Employee.company = "..."`. Prefer `ClassName.attr` when you mean the shared value.

Locals disappear when the method ends — `print(title)` after `label()` would be a `NameError`.

⚠️ WATCH OUT

```
class CartBroken:
    items = []   # WRONG — every customer shares one list

    def __init__(self, customer):
        self.customer = customer

    def add(self, sku):
        self.items.append(sku)

c1 = CartBroken("Priya")
c2 = CartBroken("Ravi")
c1.add("NB-A4")
print("Broken Ravi cart:", c2.items)
```

Expected Output:

```
Broken Ravi cart: ['NB-A4']
```

Put lists and dicts in `__init__` (`self.items = []`), not at class level, unless you truly want one shared collection.

Try It

Give `Employee` a class attribute `company = "KSR Labs"` and instance attributes `emp_id` and `name`. Print two badges, change `Employee.company`, and print again so both pick up the new value.

---

4. Instance, class, and static methods

Understand

Not every helper needs an object's data.

Kind	Decorator	First arg	Typical job
Instance	(none)	`self`	Day-to-day behaviour — `deposit`, `label`
Class	`@classmethod`	`cls`	Counters, alternate constructors (`from_row`)
Static	`@staticmethod`	neither	Pure utility namespaced on the class (GST math)

In real apps you mostly write **instance** methods. Use the other two when the intent is clear — not for decoration.

How It Works

```
class Order:
    order_count = 0

    def __init__(self, product, qty):
        self.product = product
        self.qty = qty
        Order.order_count += 1

    def line_total(self, price):          # instance method
        return self.qty * price

    @classmethod
    def get_count(cls):                   # class method
        return cls.order_count

    @classmethod
    def from_pair(cls, product, qty):     # alternate constructor
        return cls(product, qty)

    @staticmethod
    def gst_amount(amount, rate=0.18):    # static utility
        return amount * rate

o = Order.from_pair("Notebook", 3)
print("line total:", o.line_total(40))
print("order count:", Order.get_count())
print("GST on 100:", Order.gst_amount(100))
print("product:", o.product)
```

Expected Output:

```
line total: 120
order count: 1
GST on 100: 18.0
product: Notebook
```

`Order.gst_amount(100)` and `o.gst_amount(100)` both work for a static method. Prefer the class name when it is a pure utility.

Alternate constructors (`from_pair`, `from_dict`) keep `__init__` simple and give each creation path a clear name.

⚠️ WATCH OUT

A `@staticmethod` cannot read `self.qty` or `cls.order_count` unless you pass those values in. If the method needs object data, it is an instance method. If it needs the class, it is a class method.

Try It

Add `@classmethod from_csv_row` to a small `Order` (`order_id`, `sku`, `qty`) that splits `"ORD-102,PEN-01,5"`. Create one order normally, one from the CSV row, and print both labels plus a class-level count.

---

5. Encapsulation — the problem

Understand

Encapsulation means two things together:

1. **Bundle** related data and the methods that use it into one class.
2. **Hide and control** that state — outside code should change money only through a public API (`deposit`, `withdraw`, `get_balance` / `@property`) that enforces the rules.

Interview one-liner: *"Encapsulation = data + behaviour in one place, with controlled access so invariants cannot be broken from outside."*

It is **not** "putting underscores on variables." Underscores and `@property` are tools. The goal is protecting the object's rules.

Without it, any line of code can skip `withdraw` and write nonsense:

```
class OpenAccount:
    def __init__(self, holder, balance):
        self.holder = holder
        self.balance = balance   # public raw field

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds")
            return
        self.balance -= amount

acct = OpenAccount("Priya", 1000)
acct.withdraw(200)
print("after withdraw:", acct.balance)
acct.balance = -50000
acct.balance = "N/A"
print("corrupted balance:", acct.balance)
```

Expected Output:

```
after withdraw: 800
corrupted balance: N/A
```

`withdraw`'s rules are useless if callers can edit the field directly.

How It Works

Keep the real value **inside**. Expose only safe operations.

Style	Call site	Internals	Course note
Getter method	`acct.get_balance()`	often `__balance`	**Lab kit** style
`@property`	`acct.balance`	often `_balance`	Idiomatic notes / interviews

Both are acceptable. Same goal: callers who follow the API cannot casually invent `-50000`.

⚠ WATCH OUT

If the field is public, methods cannot protect it. Naming (`_` / `__`) and `@property` come next — they express this design; they are not encryption.

Try It

On paper (or in a comment): list two ways a teammate could break `OpenAccount` without calling `withdraw`. Then write one sentence: what should callers be allowed to do instead?

---

6. Public vs `_` vs `__`

Understand

Python has no true private fields. Naming tells teammates (and, for `__`, the interpreter) how to treat an attribute.

Syntax	Name people use	What Python does	What it means
`name`	**public**	Normal attribute	Part of the API — callers may read/write
`_name`	**protected** (convention)	Still a normal attribute	"Internal — please don't touch." Python does **not** block access
`__name`	**name-mangled** (often called private in notes)	Stored as `_ClassName__name`	Accidental `obj.__name` fails; determined access can still use the mangled name

None of these is security. Encapsulation is for **honest, clear code**.

How It Works

```
class AccountUnderscore:
    def __init__(self, balance):
        self._balance = float(balance)   # convention only

class AccountDunder:
    def __init__(self, balance):
        self.__balance = float(balance)  # name-mangled

u = AccountUnderscore(1000)
d = AccountDunder(1000)
```

```
print("public-style read of _balance:", u._balance)
u._balance = -50.0
print("after bad write:", u._balance)

print("has '__balance' on d?", hasattr(d, "__balance"))
print("mangled name still exists:", d._AccountDunder__balance)
```

Expected Output:

```
public-style read of _balance: 1000.0
after bad write: -50.0
has '__balance' on d? False
mangled name still exists: 1000.0
```

`print(d.__balance)` would raise `AttributeError` — the name on the object is `_AccountDunder__balance`.

⚠️ WATCH OUT

```
# Wrong — "__balance is private so nobody can read it"
print(d.__balance)                    # AttributeError (name is mangled)

# Still reachable if someone insists
print(d._AccountDunder__balance)      # works — not security
```

Leading `__` on every field is noisy. Start with public + validated methods; add `_` / `__` when the API needs it.

Try It

Create `UPIWallet` with `__balance`, `credit`, `debit`, and `get_balance()`. Credit 200, debit 100, reject an oversize debit, print the balance, and print `hasattr(wallet, "__balance")`.

---

7. `@property` and getters

Understand

A getter method (`get_balance()`) and `@property` are two public reads over hidden storage.

`@property` turns a method into something you **read like an attribute** (no parentheses): `acct.balance`.

Without a setter, `acct.balance = ...` fails — force changes through `deposit` / `withdraw`.

A setter is optional: use it when assignment should be allowed **with rules** (renaming). For bank money, prefer methods, not `acct.balance = 500`.

How It Works

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
```

```
        if amount <= 0:
            print("Invalid deposit amount")
            return
        self._balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid withdraw amount")
            return
        if amount > self._balance:
            print("Insufficient funds")
            return
        self._balance -= amount

acct = BankAccount("Priya", 1000)
acct.deposit(500)
acct.withdraw(200)
acct.withdraw(5000)
print("holder:", acct.holder)
print("balance via @property:", acct.balance)
```

Expected Output:

```
Insufficient funds
holder: Priya
balance via @property: 1300.0
```

Same idea with **lab-kit** style (`__balance` + getter):

```
class BankAccountLabStyle:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self.__balance = float(balance)

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

lab = BankAccountLabStyle("Anita", 1000)
lab.deposit(200)
print("balance via getter:", lab.get_balance())
```

Expected Output:

```
balance via getter: 1200.0
```

Optional controlled write — `@name.setter` — for fields where assignment is the natural API:

```
class Student:
    def __init__(self, name):
        self._name = name

    @property
```

```
    def name(self):
        return self._name

    @name.setter
    def name(self, value):
        value = value.strip()
        if not value:
            print("Name cannot be empty")
            return
        self._name = value

s = Student("Anita")
print(s.name)
s.name = "  Anita Sharma  "
print(s.name)
s.name = "   "
print("still:", s.name)
```

Expected Output:

```
Anita
Anita Sharma
Name cannot be empty
still: Anita Sharma
```

`@property` can also **compute** a value (`fahrenheit` from `_celsius`) without storing a second field.

⚠️ WATCH OUT

`@property` does **not** replace encapsulation by itself. Store the real value in `_…` / `__…` and validate in methods. Uncommenting `acct.balance = 999999` with no setter raises `AttributeError`.

Try It

Build a tiny `Wallet` with `_balance`, a read-only `@property` `balance`, and `deposit` only. Deposit 50 into a wallet that opened at 100 and print `w.balance`.

Step-by-Step: Build It

One assemble: a `Student` that uses instance data, a class attribute, validation, and methods together — the same skills the BankAccount kit needs, on a different domain.

```
class Student:
    college = "KSR College"

    def __init__(self, roll, name, marks):
        self.roll = roll
        self.name = name
        self.marks = list(marks)

    def average(self):
        if not self.marks:
            return 0.0
        return sum(self.marks) / len(self.marks)

    def grade(self):
        avg = self.average()
        if avg >= 85:
```

```
            return "A"
        if avg >= 70:
            return "B"
        if avg >= 50:
            return "C"
        return "Fail"

    def add_mark(self, mark):
        if mark < 0 or mark > 100:
            print("Mark must be 0–100")
            return
        self.marks.append(mark)

    def report(self):
        print(f"{self.roll} | {self.name} | {Student.college}")
        print(f"  marks={self.marks} avg={self.average():.1f} grade={self.grade()}")

s = Student(101, "Anita", [72, 68, 81])
s.add_mark(90)
s.add_mark(150)
s.report()
```

Expected Output:

```
Mark must be 0–100
101 | Anita | KSR College
  marks=[72, 68, 81, 90] avg=77.8 grade=B
```

Validation lives in methods. `college` is shared. Each student owns its own `marks` list.

Practice Exercises

Attempt each problem before reading the model answer.

1. Create `Laptop` with `brand`, `ram_gb`, and a `specs()` method that prints both. Create two objects and call `specs()` on each.

Hint: `__init__` stores the fields; `specs` uses `self`.

Model answer:

```
class Laptop:
    def __init__(self, brand, ram_gb):
        self.brand = brand
        self.ram_gb = ram_gb

    def specs(self):
        print(f"{self.brand} — {self.ram_gb} GB RAM")

l1 = Laptop("Dell", 16)
l2 = Laptop("Lenovo", 8)
l1.specs()
l2.specs()
```

Expected Output:

```
Dell — 16 GB RAM
Lenovo — 8 GB RAM
```

2. Add `upgrade_ram(extra_gb)` that increases `ram_gb` only when `extra_gb > 0`. Upgrade 8 → 16, reject `-2`, print specs.

Model answer:

```
class Laptop:
    def __init__(self, brand, ram_gb):
        self.brand = brand
        self.ram_gb = ram_gb

    def upgrade_ram(self, extra_gb):
        if extra_gb <= 0:
            print("extra_gb must be positive")
            return
        self.ram_gb += extra_gb

    def specs(self):
        print(f"{self.brand} — {self.ram_gb} GB RAM")

lap = Laptop("Dell", 8)
lap.upgrade_ram(8)
lap.upgrade_ram(-2)
lap.specs()
```

Expected Output:

```
extra_gb must be positive
Dell — 16 GB RAM
```

3. Create `Product` with `sku`, `price`, and `display()` (do **not** use `__str__` yet — that is Part 3). Add `apply_discount(percent)` that **returns** the discounted price.

Model answer:

```
class Product:
    def __init__(self, sku, price):
        self.sku = sku
        self.price = price

    def display(self):
        print(f"{self.sku} @ ₹{self.price}")

    def apply_discount(self, percent):
        return self.price * (1 - percent / 100)

Product("PEN-01", 10).display()
print(Product("X", 1000).apply_discount(10))
```

Expected Output:

```
PEN-01 @ ₹10
900.0
```

4. Create `Employee` with class attribute `company = "KSR Datavizon"` and instance attributes `emp_id`, `name`. Change `Employee.company` to `"KSR Labs"` and print both cards.

Hint: Read the shared value as `Employee.company`.

Model answer:

```
class Employee:
    company = "KSR Datavizon"

    def __init__(self, emp_id, name):
        self.emp_id = emp_id
        self.name = name

    def card(self):
        print(f"{self.emp_id}: {self.name} @ {Employee.company}")

e1 = Employee(1, "Ravi")
e2 = Employee(2, "Meera")
Employee.company = "KSR Labs"
e1.card()
e2.card()
```

Expected Output:

```
1: Ravi @ KSR Labs
2: Meera @ KSR Labs
```

5. Build a tiny `Wallet` with `_balance`, `@property` `balance`, and `deposit` only (no direct write to balance).

Model answer:

```
class Wallet:
    def __init__(self, opening=0):
        self._balance = float(opening)

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

w = Wallet(100)
w.deposit(50)
print(w.balance)
```

Expected Output:

```
150.0
```

6. Add `@staticmethod gst_amount(amount, rate=0.18)` to `Product` and print GST on ₹100 plus a line total for qty 3 at ₹40.

Hint: Static methods do not use `self`. Call `Product.gst_amount(...)`.

Model answer:

```
class Product:
    def __init__(self, sku, price):
        self.sku = sku
        self.price = price

    def line_total(self, qty):
        return self.price * qty

    @staticmethod
    def gst_amount(amount, rate=0.18):
        return amount * rate

p = Product("NB-A4", 40)
print(Product.gst_amount(100))
print(p.line_total(3))
```

Expected Output:

```
18.0
120
```

Lab

Combine a class, hidden stock, validated methods, and a class-level shop name. No files, no JSON, no `try` / `except`.

Problem

A campus store tracks one product. Stock must not go negative through the public API. The shop name is shared.

What to Build

A short script with one `Product` class and a small demo.

Requirements

- Class attribute `shop = "KSR Store"`
- `__init__(self, sku, stock)` — store stock as `__stock` (or `_stock` + `@property`)
- `sell(qty)` — reject qty ≤ 0 or qty larger than stock (print + return)
- `restock(qty)` — reject qty ≤ 0; otherwise add
- `get_stock()` or a `stock` property — return current stock
- Demo: create one product, sell, reject an oversell, restock, print shop name and stock

Expected Behavior

Oversell prints a message and leaves stock unchanged. `hasattr(p, "__stock")` is `False` if you used name mangling.

Sample Run

```
KSR Store
NB-A4: not enough stock
Stock: 12
```

(Your print wording may differ; the **rules** must match.)

Constraints

No `raise` / `try` / `except`. No files. `__str__` is optional — save it for Part 3 if you prefer a `label()` method.

Bonus

Add `@staticmethod` `gst_on(amount)` at 18% and print GST on a ₹40 item.

Mini-Project (Assignment): HDFC Bank Account Class Warm-Up

Why this matters: An account is the classic encapsulated object: the balance is not a public field; deposit and withdraw are the door.

Lab kit: 15_Lab_Assets/17_bank_account_oop/ — work in `starter/`. Do **not** copy `solution/`.

This is **not** the Student assemble from class. Same OOP tools, bank ticket. Full menu CLI stays in Part 3.

Objective

Build a `BankAccount` that deposits, withdraws, and reports balance without letting callers set the balance field directly.

Story / Scenario

An HDFC Bank campus desk needs a simple account object — rules first, menu later.

Requirements

Piece	Detail
`BankAccount`	`__init__(holder, opening=0)`; store balance as `__balance` (or `_balance` + `@property`)
`deposit(amount)`	Reject amount ≤ 0; otherwise add
`withdraw(amount)`	Reject amount ≤ 0 or overdraft; otherwise subtract
`get_balance()`	Return current balance (kit style). A `balance` property is also fine
Demo	Create one account, deposit, withdraw, print balance — **no menu loop yet**

Validate with **print + return**. Exception Handling is after Part 4.

Expected Behavior

- Positive deposit increases balance
- Overdraft and non-positive amounts print a message and leave balance unchanged
- `account.__balance` is not a casual public attribute (mangled name if you used `__balance`)

Sample Run

```
Opening: Priya @ ₹1000
After deposit 500: ₹1500.0
After withdraw 200: ₹1300.0
Overdraft attempt: Insufficient funds
```

Bonus Challenges

- Add a `@property` `balance` that reads the hidden field without a setter
- Print a one-line `status()` (save `__str__` for Part 3)

Done when

- [ ] Deposit and withdraw rules work (print + return — no `raise`)
- [ ] Balance is not casually overwritten from outside
- [ ] Kit starter ticket for the **class** portion passes
- [ ] Full CLI menu stays for Part 3

Interview Q&A

Q1. What is a class? What is an object?
A class is the blueprint (attributes + methods). An object is one concrete instance — many `BankAccount` objects from one `BankAccount` class.

Q2. What is `self` in Python?
The reference to the current instance. It must be the first parameter of instance methods; Python passes it automatically on `obj.method()`.

Q3. Why do we need `__init__`?
It sets each new object's starting state when you call `BankAccount("Priya", 1000)`. You rarely call it by name.

Q4. Instance attribute vs class attribute?
Instance attributes live on `self` (each object has its own `holder`). Class attributes live on the class (one shared `company` / `tax_rate`) unless an instance shadows the name.

Q5. `@classmethod` vs `@staticmethod`?
`@classmethod` receives `cls` — class-level data or alternate constructors. `@staticmethod` has no automatic first argument; it is a utility namespaced on the class.

Q6. What does `_balance` mean? What does `__balance` do?
`_balance` is a convention: "internal — prefer not to touch." Python does not enforce it. `__balance` is name-mangled to `_ClassName__balance`, which reduces accidental access. It is not true security.

Q7. Why use `@property`?
Expose a clean read API (`account.balance`) while keeping storage internal. Callers do not write `get_balance()` for every read. The lab kit's `get_balance()` is the same idea with a method call.

Cheat Sheet

```
class Name:              → blueprint
obj = Name(...)          → object; runs __init__
self                     → current instance (first param of instance methods)
self.x                   → instance attribute
Class.x                  → class attribute (shared)
local                    → name inside a method; dies when the method ends
def meth(self, ...):     → instance method
@classmethod             → first arg cls; counters / from_row
@staticmethod            → no self/cls; pure utility
_name                    → convention: internal
__name                   → mangled to _ClassName__name
@property                → read like an attribute (no ())
get_balance()            → lab-kit getter (same goal as @property)
```

Validate bad input with print + return in this phase — not `raise`.

Completion Checklist

```
[ ] I can define a class, create objects, and use self / __init__
[ ] I can choose instance vs class vs local variables
[ ] I can write instance, class, and static methods
[ ] I know _ is convention, __ is name mangling, and neither is security
[ ] I can hide balance and change it only through methods / a property
[ ] I completed the lab
[ ] I attempted the BankAccount warm-up (no CLI yet)
[ ] I can explain class vs object and encapsulation in an interview
```

What's Next?

You can now bundle data with behaviour and protect important state.

Next, OOP Part 2: Inheritance and Composition teaches **IS-A** reuse (`SavingsAccount` is a `BankAccount`) and **HAS-A** collaboration (an employee **has a** laptop) — without copy-pasting whole classes.